Transformer-Based Fake News Detection Using BERT and DistilBERT on the ISOT Fake and Real News Dataset

Team ID: 60

Varun Singhani (Roll No: S24BCAU0063)
Varun Jain (Roll No: S24BCAU0057)
Pranjal Gupta (Roll No: S24BCAU0048)
Shreya Mishra (Roll No: S24BCAU0049)

Department of Computer Science Engineering & Technology
Bennett University, India

Abstract

The use of news-like language to present fake news can confuse readers and erode public confidence in news reporting. Therefore, scalable screening systems are valuable. I am creating a binary text classification pipeline using BERT and DistilBERT with the ISOT Fake and Real News Dataset. In the attached notebook, I have combined each article's title and body, converted the combined text to lowercase, removed punctuation and duplicate whitespaces, removed duplicate normalized articles, randomly sampled 8,000 records from the dataset at a specified sampling seed. I then set up an 80/20 GroupShuffleSplit with the normalized text as the group variable. I tokenize the text to a max length of 256 and fine-tune the pre-trained transformer checkpoints for two epochs with a learning rate of 3x10^-5 and a batch size of eight. When evaluated directly against BERT with 1,600 test articles, BERT produces 99.8125% accuracy, 99.7745% precision, 99.8871% recall and 99.8308% F1 for label 1 (real news). BERT has produced only three errors out of 1,600 examples in its confusion matrix. A direct DistilBERT metric dictionary was not available in the saved output files, however, I did find a summary cell later on in the notebook that reported inconsistent near-perfect scores from an unknown results variable; these results were therefore considered unverified. My primary contribution is a repeatable pipeline design and a clear analysis of how well each model performed and the limits of the available experimental record.

Index Terms—*fake news detection, BERT, Distil BERT, transformer models, ISOT dataset, text classification*

I. INTRODUCTION

While online news and social media allow individuals to easily disseminate and share information, the speed at which this happens can also provide an avenue for false or misleading stories to be disseminated quickly. Reliable reporting typically does not disseminate in the same manner as reliable news stories. Reliable reporting has a positive impact on the way people engage in civic discussions, the level of trust they have in sources of information, and the decision making process of government officials [16]–[18]. At the present time, manual verification of information remains a necessary step; however, verifying all of the articles being shared is virtually impossible due to the sheer volume of content being produced. Automated classifiers can assist with identifying content that needs to undergo verification prior to being shared, as long as users understand that automated classifier predictions are not equivalent to a third party verifying the accuracy of the information contained within the article.

The detection of fake news is considered as a supervised learning problem because there are labeled examples (e.g., fake vs real). Early research in this area typically relied on lexical counts, manually created writing style features, source credibility, and/or propagation patterns. While these features are potentially valuable, they do not necessarily reflect the relationship among different parts of a long article. Transformer encoders provide contextual representations that can be fine-tuned for classification tasks using relatively simple task-specific architectures [1]. DistilBERT reduces both the size of the BERT family models and the time it takes to perform inferences with them, while maintaining a significant portion of their language understanding capabilities [2]. This combination makes BERT and DistilBERT good models to compare for a text-classification study.

The ISOT Fake and Real News dataset is used for this project; the real-news articles were extracted from Reuters while the fake-news articles were extracted from sources that have been labeled as "unreliable" [15]. While this collection may be useful for conducting controlled experiments, the differences between the two classes may relate to other factors such as publisher, topic, date, and writing style. Therefore, a model achieving high test accuracy using this dataset will not provide evidence that it is capable of identifying fraudulent claims in news articles that come from unknown sources. Additionally, both generalization and dataset biases are well-known issues within the field of natural language processing [3], [14].

The issue being studied in this research is to categorize a user-provided English language news story as either false (labeled 0) or true (labeled 1) based on the title and content of the article. There are three key contributions of the research: (1) we have documented a process for pre-processing and creating group splits for 8,000 articles, (2) we have trained both BERT and DistilBERT with the same training settings, and (3) we have reported the direct metrics of BERT, the confusion matrix, and the gaps in evidence which limit our ability to perform a direct comparison between the two models. The main goal of this research is to assist humans in making decisions about whether to trust a given news source, rather than replacing them with automated systems that can determine if a news source is trustworthy or blacklisting publishers.

The goal of operation is to learn a function f(x) that takes an input x which is a normalized string of an article, and produces a binary output y (0 or 1). An output of 0 indicates that the article is similar to the fake articles in our dataset. An output of 1 indicates that the article is similar to the real articles in our dataset. This difference is important because the labels represent annotation on the dataset rather than a new search for the truth of each individual sentence. If you were doing a real verification process, then a high fake-class score would indicate you need to review the source, cited evidence, publication date, and any other relevant independent reporting. Conversely, a high real-class score does not prove that the article is correct.

II. LITERATURE SURVEY

Zhou and Zafarani reviewed fake-news detection by looking at different aspects including: false knowledge, writing styles, propagation patterns, and sources' credibility [3]. They developed a taxonomy that illustrates why text-only models are just one aspect of verifying information. Similarly, Shu et al. discussed the limitations of relying upon social engagement signals due to their incompleteness, noisiness, and reliance on the way in which a story spreads [4]. FakeNewsNet was developed to allow researchers to combine article content with social contexts and temporal data [5]. Based on these references, it appears that deploying a reliable solution will likely need additional features besides the wording of a single article.

The Fakeddit dataset was developed by Nakamura et al., a large multi-modal data set that includes text, images, metadata, comment data, and several levels of labeling [6]. This wide variety of data types is intended to overcome some of the limitations of using only text-based data, but the use of distant supervision and other types of data provide additional complications. For this study, we intentionally used only the textual data from the ISOT dataset so we could easily manage the pipeline and therefore limit the types of evidence that would be presented to the classifier.

In addition, Devlin et al. developed the bidirectional pre-training of BERT and demonstrated that it could be used for other downstream NLP tasks through fine-tuning [1]. DistilBERT was created by Sanh et al. as a way to decrease the model size of BERT and increase the speed of inferencing via knowledge distillation [2]. FakeBERT is an architecture created by Kaliyar et al., which includes both BERT and multiple CNN layers to identify local text features. FakeBERT achieved 98.90% accuracy in its experiments [7]. Low et al. examined how difficult it would be to distinguish between fake news and satire, noting that transformer design and domain adaptation pre-training play a role in distinguishing between classes that have similar styles [8]. Based on these previous works, we plan to compare a large encoder-based model to a smaller distilled version of the same model; however, we will note that labels may vary depending on the specific task.

The authors Babu et al., evaluated transformers on three different types of data (LIAR, FNC-1, and a balanced fake news dataset) and found that each model performed differently on the various datasets; also, the author's DistilBERT model was able to produce results faster than other models, demonstrating that the order in which models rank depends upon the specific type of data used for testing [9]. The authors Alghamdi et al., provided an overview of different types of ML approaches (content-based, context-based, and hybrid) and described some of the limitations of common evaluation datasets and protocols [11]. The authors Athira et al., investigated explaining why models make certain predictions and stated that there is a discrepancy between achieving high levels of accuracy in predicting outcomes and providing explanations that can be viewed and interpreted by people [10]. Although SHAP is loaded into the provided notebook, there are no SHAP values generated nor are there any SHAP plots created when running the code in the notebook.

The dataset problem is even clearer now than before. Kuntur et al. used the same transformers and graph neural networks to test FakeNewsNet, ISOT and WELFake and saw significantly different performance across each dataset. They reported near-perfect transformer results on certain corpus, while reporting poor performance on FakeNewsNet [12]. Bazmi et al., developed an entity-centric multi-domain transformer model designed to improve models ability to generalize to new news domains. Their zero-shot testing shows that models that perform well in a particular domain will not necessarily perform as well when tested in other domains [13]. Kishi et al. (2026) studied dataset bias across many datasets and demonstrated that the effect of a training strategy on in-dataset performance and cross-dataset generalization may differ [14].

These studies collectively imply that results need to be interpreted using three levels. The first level is called "in-domain" discrimination, which determines if a model accurately reproduces the labels of held out articles within a known collection. The second level is "robustness", which determines if a model performs well regardless of changes in publishers, topics, time frames, and labeling methods. Finally, the third level is "usefulness for human review," and this relates to how calibrated the model is; how much money it costs when it makes errors; and how easily it can explain to humans why it was unsure about its prediction. In this notebook, I primarily address the first level. A confusion matrix allows me to see how many times the model made mistakes, however, I did not perform a "leave-publisher-out" test, cross-dataset evaluations, probability calibrations or user facing explanations. These are not small details of implementing the model, rather they will determine if the accuracy reported transfers to the constantly evolving news ecosystem.

Prior research provides powerful architectures and richer datasets, however performance may be overstated if labels correlate with source specific features, or if experimental records are not reproducible. This study develops a compact BERT/DistilBERT architecture based on ISOT, and emphasizes evaluation traceability. To date, there is no evidence to demonstrate that the model will perform well across multiple sources, as we have not developed a separately verified validation set, a controlled baseline using the same split, nor a complete DistilBERT metric log are absent.

III. PROPOSED METHODOLOGY

A. System Architecture

Figure 1 shows the pipeline. It reads the CSV files, concatenates the titles of each file with the body of that file, removes duplicate normalized texts, and creates a fixed size random sample. After that it defines a group aware 80/20 train/test split. Then, it uses two different pre-trained tokenizers to tokenize the exact same text. Next, it passes those token IDs and attention masks into BERT base uncased and DistilBERT base uncased sequence classification models. Finally, both models emit logits for the two labels; the predicted class is the index of the highest logit. The notebook also calculates accuracy, precision, recall, F1, and a BERT confusion matrix.

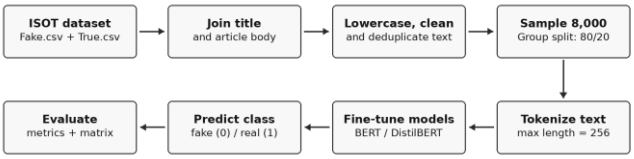


Fig. 1. End-to-end text classification workflow implemented in the notebook.

B. Dataset and Preprocessing

The source collection includes True.csv and Fake.csv. The notebook will assign a value of 0 to fake articles and 1 to real articles, combine them into one DataFrame, keep only the text column, and add the label. Text cleaning involves converting each character to lowercase, reducing the amount of whitespace, using a regular expression to eliminate all punctuation, removing leading and trailing whitespace, and discarding any identical normalized text entries. After completing these steps, the code will sample 8,000 observations using a random_state value of 42. When we review the final DataFrame shape in our output, we can see that it has 8,000 rows and two columns [15]. We do not maintain the exact number of classes for the evaluation of the confusion matrix test subset. This subset contains 714 fake articles and 886 real articles, therefore the test subset used for evaluating the model is not perfectly balanced.

TABLE I. DATASET AND PREPROCESSING SUMMARY

Item	Implementation
Source	ISOT Kaggle mirror: Fake.csv and True.csv
Labels	0 = fake; 1 = real
Input	Title + article body
Cleaning	Lowercase, whitespace normalization, punctuation removal
Duplicates	Drop duplicate normalized text
Sample	8,000 articles; random_state = 42
Split	80% train / 20% test; GroupShuffleSplit

The GroupShuffleSplit was set up with a test_size of.20, random_state of 42, and the articles' normalized texts as groups. It provides some level of protection from having identical texts in both sides of the split. Prior to sampling, all exact duplicate articles are removed. However, the separate intersection check performed in the notebook is done before the visible cell where the current train/test lists were created. Therefore, the zero-overlap result reported by the notebook is not a guarantee of the split that was created by the later

cell. The notebook needs to be run again from the first cell, and the overlap assertion must be executed immediately after the train/test split.

An 8,000-article sample size creates an appropriate amount of training data for fine-tuning while creating a manageable test set of 1,600 examples. However, by selecting a subset of the entire source corpus, some of the possible information is being discarded and the selection process creates sampling error. A single randomly selected sample does not allow us to determine how dependent the outcome is upon the initial seed selection. The splitting routine is aware of groups (articles) but is not explicitly stratifying based on classes. This is evidenced by the displayed test matrix which contains 714 examples labeled as '0' and 886 examples labeled as '1.' For a more rigorous evaluation, the number of examples in each class within the train, validation, and test sets should be documented, groups should be formed using a consistent identifier for each article/source (where possible), and multiple random seeds should be compared. Additionally, original article and publisher metadata should remain intact as separate entities for auditing purposes, even if only the article text will be used as input for classification.

C. Tokenization and Model Design

The model tokenizer will convert the text into encoded forms (with truncation, padding and max_length=256) of input data. The PyTorch custom dataset will output the encoded inputs (as tensors) and corresponding label integers. BERT-base-uncased has 12 transformer layers, which equates to about 110 million parameters [1]. DistilBERT-base-uncased contains 6 transformer layers, equating to roughly 66-67 million parameters [2]. Each of the two models was fine-tuned on the same split of the data and utilizes a two-label sequence-classification head that was initially set up specifically for this task. It is anticipated that the smaller model will decrease both memory and latency requirements; however, it is unknown if the smaller model actually decreased either parameter as the saved notebook did not provide any per-model run time or DistilBERT performance metrics.

Tokenizing is the link between raw news text and the numerical input to the model. Tokenizers have their own specific token ID's and attention masks that they create from the raw news text. Padding is used so that we can use different sized articles when we run them in batches. Articles longer than 256 tokens will be cut off at that point. Since the title and body of the article are combined together prior to being tokenized, most likely, the title will appear near the beginning of the article, while some parts of the article may get cut off if it is very long. Therefore, the length of the article becomes an important variable in our experiments, since this determines how much information our classifier gets to "see," and also how much memory our experiment requires. The notebook does not evaluate the effects of changing the sequence length, keeping the title and body separate, or removing punctuation to see if those would improve our results. We need to evaluate these variables as ablation studies on a separate validation set, rather than simply assume they are best. A subsequent experiment can evaluate the differences between using only titles as inputs, only bodies as inputs, and both titles and bodies as inputs; as well as evaluate the effects of a few different sequence lengths (on the exact same split) to assess whether additional computing power provides a noticeable improvement in performance.

D. Training Configuration and Optimization

For both calls, the learning rate will be 3×10^{-5}, per-device training and evaluation batch sizes will be eight, there will be two epochs, mixed precision (fp16=True) will be used, logging will occur every 100 steps, no model checkpoint saving will occur, and external reporting will be disabled. The Hugging Face Trainer API handles batching, gradient updates, and sequence classification loss. The model produces logits z_0 and z_1 when predicting for a two class output, and selects $\hat{y} = \text{argmax}(z_0, z_1)$. Cross entropy is the standard classification objective used by the sequence classification head. The notebook did not specify a separate held out validation partition or conduct a parameter search; it utilized one fixed configuration for each model. Therefore, this report does not state that 3×10^{-5}, batch size 8, or two epochs were demonstrated to be optimal.

The saved predict helper can accept either a single string or multiple strings, then tokenize each string using both padding and truncation, transfer the resulting tensor(s) onto the device where the trained BERT model resides, disable the gradient calculation, and apply the softmax function to the logits. Softmax transforms the two logits into a pair of numbers whose sum is equal to 1. However, these pairs of numbers do not represent calibrated probabilities. The helper provided in the notebook was written specifically for BERT; a comparable DistilBERT helper would utilize the DistilBERT tokenizer and trained DistilBERT model if you wanted to expose both models under a single API. Also, since we used save_strategy="no", our trained model weights will not persist after training via Trainer checkpoints. Therefore, to obtain our trained weights again (before deploying or auditing independently), we will need to re-run the training process from scratch.

TABLE II. SHARED TRAINING CONFIGURATION

Parameter	Value in code
Base checkpoints	bert-base-uncased; distilbert-base-uncased
Learning rate	3×10^{-5}
Batch size	8 train / 8 evaluation
Epochs	2
Maximum sequence length	256 tokens
Precision	FP16 enabled
Checkpoint saving	Disabled
Split/sample seed	42 for sampling and split

E. Formal Objective and Prediction Rule

Let x_i represent the normalized title-and-body text from article i, and let y_i\in\{0,1\} represent the dataset label for article i (y_i=0 if the article is fake, y_i=1 if the article is real). Tokenization translates x_i into a bounded series of token ID numbers and attention masks. A sequence-classification model with parameters θ will produce two logit scores, one for each class. Softmax will translate the two logit scores into class probabilities p_{i,c}. The class probability that has the highest value will be used as the predicted label for article i. While fine-tuning, the classifier will minimize cross-entropy across all training examples. The equations below specify the standard two-class objective function related to the model families being studied; they define the problem to be solved and do not imply that other experiments were conducted.

$$p_{i,c} = \exp(z_{i,c}) / [\exp(z_{i,0}) + \exp(z_{i,1})], \quad c \in \{0, 1\} \quad (1)$$

$$L(\theta) = -(1/N) \sum_{i=1}^{N} \sum_{c=0}^{1} y_{i,c} \log(p_{i,c}), \quad \hat{y}_i = \arg\max_c p_{i,c} \quad (2)$$

The formula for calculating the logits of each of the classes (c) for each of the instances (i) in the dataset is represented by z_i,c; while the softmax score of each of the classes for each of the instances in the dataset is represented by p_i,c; and the one-hot target indicator for each of the classes for each of the instances in the dataset is represented by y_i,c. The total number of training examples is represented by N. The predicted class for each of the instances in the dataset is represented by $\hat{y}_i$. When evaluating model performance, we compare the predicted labels from our model with the true labels provided in the ISOT dataset to determine if they match, and we use these comparisons to generate confusion matrices and other classification metrics. A correct prediction does not mean that the model has independently verified the factual claims made within an article, but rather that the model agrees with the ISOT annotation.

IV. EXPERIMENTAL RESULTS

A. Computational Environment

The output of the saved installation provides information on what was available at that time (Python 3.12 environment, Transformers 5.7.0, and PyTorch 2.10.0+cu128). This indicates that the training arguments were set to use FP16 which would allow the code to run on a CUDA enabled device. However, there are no specific details regarding the exact GPU used during the execution of the code, nor does it list the CPU type or amount of RAM that was available. In order to capture these types of details in subsequent experiments, it would be beneficial to include them within the code or documentation. In addition to importing Pandas, NumPy, scikit-learn, Matplotlib, Seaborn, WordCloud, SHAP, PyTorch, and Hugging Face Transformers; the use of CountVectorizer is employed to illustrate frequent bigrams. There is no classical classification algorithm such as Logistic Regression or Support Vector Machines (SVM) trained in the provided notebook.

B. Evaluation Metrics

Accuracy represents the percentage of correctly identified instances: (TP+TN)/(TP+TN+FP+FN). Precision is TP/(TP+FP), which represents the percentage of identified positive cases that are correct. Recall is TP/(TP+FN), representing the percentage of true positive cases identified. F1 is a combination of precision and recall; it is calculated by taking the average of the reciprocals of precision and recall. Since the compute_metrics function in the notebook utilizes scikit-learn's default average='binary' setting without specifying a value for pos_label, the model treats label 1 (real news) as the positive class. Thus, the metrics presented below should not be referred to as "fake news precision/recall."

C. Direct BERT Test Results

The explicit output of trainer_bert.evaluate() states eval_accuracy=0.998125, eval_precision=0.997745, eval_recall=0.998871, and eval_f1=0.998308. There are 1,600 articles in the test set. According to the BERT confusion matrix displayed in the notebook there were 712 fake articles that were accurately identified as fake; two fake articles that were incorrectly identified as being real; one real article that was incorrectly identified as being fake; and 885 real articles that were correctly identified. Therefore, out of 1,600 total articles tested, 1,597 were accurately identified, which is consistent with the 99.8125% accuracy rate reported.

The corresponding total error rate was 3/1,600 = 0.1875%. In the case of the fake class (directly obtained from the confusion matrix), precision is 712/713 = 0.9986, recall is 712/714 = 0.9972, and F1 is 0.9979. For the real class, the direct Trainer output produced precision 0.9977, recall 0.9989, and F1 0.9983. Because of the slightly lower fake-class recall, there were two fake labeled articles that were not detected; therefore, for a screening tool designed to detect potentially misleading articles, both types of false negative should receive some consideration, despite the overall high accuracy of the system. Conversely, one real article was misclassified as fake, showing the other side of the trade-off -- either the unnecessary human review of the article, or the damage to the reputation of the article if it were accepted as fact based on the system's prediction.

TABLE III. CONFUSION MATRIX FOR BERT (ROWS = ACTUAL; COLUMNS = PREDICTED)

Actual class	Predicted fake	Predicted real	Total
Fake (0)	712	2	714
Real (1)	1	885	886
Total	713	887	1,600

TABLE IV. DIRECT BERT METRICS AND TRACEABILITY STATUS

Model/output	Accuracy	Precision	Recall	F1
BERT evaluate()	0.9981	0.9977	0.9989	0.9983
DistilBERT evaluate()	Not printed	Not printed	Not printed	Not printed

The saved notebook includes another dataframe later which displays BERT accuracy as 0.999889 and DistilBERT accuracy as 1.000000. These cells are visible, however, they do not show the results dictionary that created the dataframe; therefore the BERT accuracy is conflicting with the direct evaluation output provided earlier. These accuracies will not be considered verified results. Although the training calls for both DistilBERT and BERT, and the evaluate() calls for both are shown in the notebook, the numerical evaluation dictionary for either has not been displayed in the saved output. From these outputs alone, it is impossible to determine a final model ranking.

In this document I am explicitly making a distinction between the results of my experiments that have a trace back to their origin and those that are "orphan" summary results. For BERT, this distinction may seem insignificant, however the inability to trace back the source of the summary means I cannot tell if the summary was generated on the same sample, a previous model checkpoint, or a completely different experiment. If we were looking at perfect accuracy with DistilBERT, we would need to inspect each prediction to determine if the 1.0000 was due to shortcut features in the dataset, or a truly discriminative representation. Therefore, instead of selecting the more favorable number, we should rerun the notebook in a new session, print out results_bert, results_distil, each confusion matrix, and the test indices from the same run.

D. Comparison with Prior Work

Published findings give some context; however, due to differences in dataset, label, training/testing protocol, pre-processing, and computation environment, they cannot serve as a controlled baseline. FakeBERT achieved 98.90% accuracy in its experimental setup [7]. Babu et al. showed that RoBERTa performed well on FNC-1 yet had lower performance on LIAR. This demonstrates the impact of both the difficulty of the corpus and the structure of the labels on performance [9]. Kuntur et al. found that transformers performed very well on ISOT and WELFake, although performance was significantly worse on FakeNewsNet. Kuntur et al. highlighted that it is important to verify findings across multiple datasets [12]. In the prior project report, 98% BERT accuracy/F1 and 95% DistilBERT accuracy/F1 were reported for a different configuration that used more data and three epochs. These results are kept as prior project context and will not be combined with the 8,000-record notebook run.

TABLE V. CONTEXTUAL COMPARISON WITH PUBLISHED AND PRIOR-REPORT RESULTS

Study / setup	Dataset	Reported result	Interpretation
FakeBERT [7]	Study setting	98.90% accuracy	Different architecture and protocol
Babu et al. [9]	FNC-1	RoBERTa: 97% accuracy	Not same dataset
Kuntur et al. [12]	ISOT	RoBERTa: 99.99% accuracy	Reimplementation; source-bias caveat

Study / setup	Dataset	Reported result	Interpretation
Earlier report	ISOT, larger run	BERT 98%; DistilBERT 95%	Different training configuration
Current direct output	ISOT sample, n=8,000	BERT: 99.8125% accuracy	Direct evaluate() output

Therefore, this current notebook illustrates how well BERT performs on classifying the labels in this test subset with very little error. This performance is an indication of in-dataset classification capabilities, rather than a verification of "real world" truth detection. The class labels are based on the dataset collection process; language, topics, and dates specific to each publisher could be used as a shortcut. Cross-source and cross-dataset testing will be necessary before making the claim that the model generalizes [12]-[14].

The previously submitted report and current notebook should not be considered "repeated" runs of a single experiment; they are two different experiments. The previous report used a larger ISOT corpus and ran for three training epochs. The attached notebook specifically selects 8,000 records from the ISOT corpus and trains on those records for two epochs. It is possible that the differences in sample size, selection of records, preprocessing of records, and training of records may lead to varying levels of difficulty of testing examples, and also to variations in how the decision boundaries are formed by each model. Therefore, prior to making a definitive statement about the superiority of BERT vs. DistilBERT, it will be necessary to utilize the same split for both models (both class-wise and macro metric calculations) within a single script. Additionally, it will be beneficial to report the runtime and parameter count of each model along with its accuracy. Finally, a basic TF-IDF logistic regression baseline and a linear support vector machine (SVM) will assist in determining whether the high score achieved by both models is due to the contextual semantic representation of the data or to lexical/source-related artifacts of the data.

A fair numerical comparison will separate the effect of architecture from that of different experimental procedures. The earlier report had 98% BERT and 95% DistilBERT based upon a larger data set and three epochs. For the current study we sample 8,000 examples and use two epochs. As such, these two figures should not be interpreted as evidence that one model has either improved or declined, unless they are rerun using the same protocol. This is true also for reports in the literature; while reporting a figure for FNC-1, WELFake, or a custom dataset provides useful context, variations in label taxonomy and source material make it impossible to rank models directly. Controlled benchmarks need to specify which version of the dataset was used, how many samples were used, how the data was split, how classes were defined, which model checkpoint was used, and if scores were obtained from an independent held-out set. Given this, the only valid comparison that can be made is between the currently logged BERT result and the earlier report's contextual information, and we must explicitly state that the DistilBERT results remain unverified.

E. Hyperparameter Tuning and Reproducibility

The provided code does not use either grid search or random search to find optimal hyper-parameters; instead, it uses a single hyper-parameter combination. There are no tables of validation accuracy for other learning rates, batch sizes, sequence lengths, and/or number of epochs. Therefore, the hyper-parameters given in Table II represent those used to create the model, not an empirical optimization of these hyper-parameters. Additionally, the notebook disables model checkpointing and does not export the learned weights or predictions as stable artifacts. To produce a reproducible benchmark, the notebook needs to be executed using a new kernel, both models need to have their evaluation dictionaries preserved, confusion matrices need to be saved for both models, and package versions and hardware characteristics need to be recorded. Finally, a validation set should be created from the training data, and the final test set should remain unperturbed until the chosen configuration is locked down.

The proposed next step for this study is to evaluate the effect of varying hyperparameter settings on the model's performance. Specifically, the experiment will assess how the learning rate (i.e., 2x10^-5, 3x10^-5, and 5x10^-5), batch size (i.e., 8 and 16, subject to available memory), and number of epochs (i.e., 2 vs. 3) affect model performance. Hyperparameters that result in optimal performance will be selected based on the validation macro-F1 score or fake-class recall as specified in the review policy, and not through multiple evaluations of the test set. After selecting the optimal hyperparameters, the final test set evaluations will be performed. This proposed future research is not part of the experiments reported in the present notebook. Additionally, to ensure reproducibility, the exact version of the Kaggle dataset used, along with the list of randomly sampled article identifiers should be documented in addition to setting random_state=42. Since Kaggle datasets may be updated independently from the code, documentation of the dataset version and randomly sampled article identifier lists are essential for replicating the results.

F. Limitations and Responsible Use

The dataset is composed of English-language content that was primarily collected in 2016-2017 [15] as political/world news. The labels are binary (they indicate whether something is either true or false) and do not reflect any degree of uncertainty, satire, misleading framing, or partial factual errors. The experiment will be conducted using only text; images, external links, author information, and social network dissemination characteristics will not be utilized. Although SHAP is imported into the program, it is not run. Therefore, the system will not provide explanation of which words contributed to an individual prediction. Additionally, the split-overlap check that occurs prior to the later visible split cell is not included in the visible output, nor is DistilBERT's numeric evaluation. Lastly, due to both the extreme class imbalance in the test subset and the exceptionally high in-domain score, it would be beneficial to report on a per-class basis and to evaluate the classifier externally. This classifier should be employed to identify articles that require human verification, rather than as a primary source of information for moderating content or asserting factual certainty.

There are other limitations to our study, such as the possibility of removing stylistic cues with our use of punctuation removal and low-case conversion; and the potential loss of evidence located near the end of a long article due to our token-truncation strategy (at 256 tokens). Since we concatenate the title and body prior to token-truncation, it is highly probable that the title will be preserved; however, long bodies may be cut-off. In addition, our source label approach could result in the model learning to identify the outlet identity of a claim rather than verifying whether a claim is valid. To develop a stronger study, we should examine how well the model performs without using publisher identifiers or holding-out those identifiers, use more recent reports and diverse topics, and manually review a representative subset of both false-positives and false-negatives. Additionally, we would need to establish confidence levels, abstain from classifying examples that we are uncertain about, and provide citations for supporting evidence before we can consider building an operational system that can operate responsibly..

G. Error Analysis and Decision Thresholds

The confusion matrix provides a way to examine the types of errors made in each category (i.e., fake and real) as opposed to having them buried within an overall accuracy score. Fake-class recall was 99.72% since 712 of the 714 fake-labeled articles were found to be fake and only 2 were identified as real. Real-class recall

was 99.89% since 885 out of the 886 real-labeled articles were identified as real and only 1 was identified as fake. Fake-class precision is 99.86% and real-class precision is 99.77%. These precision scores are based on the saved matrix and not a separate evaluation. Accuracy scores would obscure the fact that there are still some false negative cases in the test set since there are more real articles than fake ones in the test set. When reviewing stories for a screening workflow, there are differences in terms of the potential damage caused by missing a misleading story versus unnecessarily flagging a reliable story. Therefore, selecting the most appropriate operating point depends upon what is most important to consider (i.e., recall, precision, available reviewers, reputation).

In the notebook, the prediction helper utilizes the argmax operation on the two class logits after applying the softmax function. This results in a class label for each instance; however, there is no defined decision threshold based on how many cases are reviewed by humans, and it cannot be assumed that the softmax scores are calibrated probabilities. In a future study, it will be necessary to collect class scores for all test instances, create precision-recall curves for each class, assess the calibration of these scores, and determine a decision threshold using a validation set. Furthermore, it will be required to provide precision and recall for both classes, calculate macro-F1, and display the confusion matrix for each architecture. A decision threshold may then be determined based upon a predetermined objective (e.g., maximizing the recall of the fake class while minimizing the number of articles sent to human reviewers). At this time, neither threshold selection nor calibration experiments have been documented within the notebook [10], [13].

H. Generalization and Follow-up Evaluation

The subsequent evaluation will determine if the classifier has developed generalizable indicators of deception, as opposed to characteristics that are unique to the particular set of articles used for training. To achieve this, several steps can be taken: 1) whenever possible based upon available source metadata, retain all articles from a specific publisher/domain on only one side of the split; a publisher-held-out test is more stringent than randomly dividing each article. 2) conduct a chronological test, whereby the training examples precede the evaluation period; this is necessary because language usage, subject matter, and reporting styles vary over time. 3) evaluate the classifier with no modifications on at least one distinct dataset (e.g., FakeNewsNet or WELFake) that can be mapped to the binary task. These evaluations should be reported separately, as opposed to combining them with the ISOT evaluations due to the variations in the underlying datasets and label definitions [5], [12], [13].

The experiment needs to store the identifier for each evaluated dataset; the index of each train/validation/test split; the version number of all packages used; the name of each checkpoint; the seed used for randomness; the number of classes present in each dataset; the output of the model (logit values); and the full dictionary containing all calculated metrics. Both BERT and DistilBERT must have identical train/validation/test split definitions. A simple TF-IDF based model must be trained on the exact same splits as well to determine how much of a performance improvement is achieved through using a contextual encoder. A manual analysis of both false positives and false negatives could potentially reveal if there were common themes among them (e.g., were some of the articles satirical? Were many of the articles outdated? Was there evidence that misleading headlines contributed to misclassification? Were there article-level source cues that could have influenced classification?). These types of analyses were not conducted in the present study. They are suggested as possible controls for future studies, which will likely be more robust than the present study. This multi-stage approach will allow us to create a more defensible comparative study, while maintaining a reasonable level of modesty regarding the final goal of the application: to prioritize stories for review by people who provide evidence-based reviews of those stories, as opposed to providing an absolute classification of whether an article is either true or false.

I. Reporting Completeness and Interpretation

A machine-learning claim is as good as the ability to show how it was produced by a specific split of data, followed by showing how that split led to a certain set of predictions. BERT's current output allows for auditing; there were 1,600 examples in total within the confusion matrix; there were 1,597 correct predictions in the diagonal of the confusion matrix, which yields an overall accuracy of 0.998125. The class-specific values generated from that matrix are beneficial since the metric function considers label 1 (real news) as the positive class by default. For this report, the precision and recall for fake-class will be obtained directly from the matrix rather than incorrectly copying them from the default binary metric output. Thus, the label convention is made evident, preventing the metrics of one class from being treated as if they represented those of the other class.

The evidence remains limited. One hold-out partition provides only an estimate based on one sample and one split; the saved notebook does not provide information about variability across different seeds, confidence intervals, or a separate validation dataset. The DistilBERT numerical metric dictionary was not included in the saved output, and the later summary dataframe did not match the direct BERT metric. To release a verified version, the training script must save the sampled rows IDs, the train/test indices, the model checkpoint ID, the evaluation dictionary, the class-by-class report, the confusion matrix, and the prediction scores for both architectures into permanent storage. Then, using those same saved artifacts, regenerate the report. That audit trail will enable the team to reproduce each table, determine if there is indeed no overlap between the groups being split, and compare the cost and performance of each model without having to rely on the display state of the notebook. Those artifacts are recommended for future runs, not assumed to exist in this submission.

V. CONCLUSION

This project uses a transformer-based fake-news classification pipeline that employs BERT and DistilBERT with the ISOT Fake and Real News dataset. The notebook preprocesses the text in each article; removes duplicate articles; samples 8,000 articles; splits the sampled articles into an 80/20 train/test split based on group awareness; tokenizes the sampled articles to 256 tokens; and fine-tunes each model for two epochs. BERT's evaluation result of 99.8125% accuracy and 0.9983 F1 for label 1 (real news), with three errors in a 1,600-article test set, was directly recorded. However, a DistilBERT score can't be verified from the saved numeric outputs, and the later near-perfect comparison cell isn't consistent with the direct evaluation log. Based upon these results, it can be concluded that while the models performed well on the ISOT test subset, they aren't necessarily capable of detecting misinformation in real-world environments. Additional future work includes: re-running and saving both evaluation outputs; evaluating the models on source-held-out and external datasets; comparing traditional baseline approaches on the same split; tuning hyper-parameters using a separate validation set; and implementing explainability and calibration techniques to facilitate human fact-checking.

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. 2019 Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.

[2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” arXiv:1910.01108, 2019.

[3] X. Zhou and R. Zafarani, “A survey of fake news: Fundamental theories, detection methods, and opportunities,” ACM Comput. Surv., vol. 53, no. 5, Art. no. 109, pp. 1–40, 2020, doi: 10.1145/3395046.

[4] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, “Fake news detection on social media: A data mining perspective,” ACM SIGKDD Explor. Newsl., vol. 19, no. 1, pp. 22–36, 2017, doi: 10.1145/3137597.3137600.

[5] K. Shu, D. Mahudeswaran, S. Wang, D. Lee, and H. Liu, “FakeNewsNet: A data repository with news content, social context, and spatiotemporal information for studying fake news on social media,” Big Data, vol. 8, no. 3, pp. 171–188, 2020, doi: 10.1089/big.2020.0062.

[6] K. Nakamura, S. Levy, and W. Y. Wang, “Fakeddit: A new multimodal benchmark dataset for fine-grained fake news detection,” in Proc. 12th Language Resources and Evaluation Conf. (LREC), Marseille, France, 2020, pp. 6149–6157.

[7] R. K. Kaliyar, A. Goswami, and P. Narang, “FakeBERT: Fake news detection in social media with a BERT-based deep learning approach,” Multimedia Tools Appl., vol. 80, no. 8, pp. 11765–11788, 2021, doi: 10.1007/s11042-020-10183-2.

[8] J. F. Low, B. C. M. Fung, F. Iqbal, and S.-C. Huang, “Distinguishing between fake news and satire with transformers,” Expert Syst. Appl., vol. 187, Art. no. 115824, 2022, doi: 10.1016/j.eswa.2021.115824.

[9] R. N. Babu, C.-H. Lung, and M. Zaman, “Performance evaluation of transformer-based NLP models on fake news detection datasets,” in Proc. IEEE 47th Annual Computers, Software, and Applications Conf. (COMPSAC), 2023, doi: 10.1109/COMPSAC57700.2023.00049.

[10] A. B. Athira, S. D. M. Kumar, and A. M. Chacko, “A systematic survey on explainable AI applied to fake news detection,” Eng. Appl. Artif. Intell., vol. 122, Art. no. 106087, 2023, doi: 10.1016/j.engappai.2023.106087.

[11] J. Alghamdi, S. Luo, and Y. Lin, “A comprehensive survey on machine learning approaches for fake news detection,” Multimedia Tools Appl., vol. 83, pp. 51009–51067, 2024, doi: 10.1007/s11042-023-17470-8.

[12] S. Kuntur, M. Krzywda, A. Wróblewska, M. Paprzycki, and M. Ganzha, “Comparative analysis of graph neural networks and transformers for robust fake news detection: A verification and reimplementation study,” Electronics, vol. 13, no. 23, Art. no. 4784, 2024, doi: 10.3390/electronics13234784.

[13] P. Bazmi, M. Asadpour, A. Shakery, and A. Maazallahi, “Entity-centric multi-domain transformer for improving generalization in fake news detection,” Inf. Process. Manag., vol. 61, no. 5, Art. no. 103807, 2024, doi: 10.1016/j.ipm.2024.103807.

[14] Y. Kishi, Y. Arima, and H. Iyatomi, “Fake news detection strategies under dataset bias: Using large-scale coarse-grained labels,” in Proc. 19th Conf. European Chapter of the Association for Computational Linguistics, Student Research Workshop, Rabat, Morocco, 2026, pp. 612–621, doi: 10.18653/v1/2026.eacl-srw.47.

[15] ISOT Research Group, University of Victoria, “ISOT Fake News Dataset: ReadMe,” 2023. [Online]. Available: https://onlineacademiccommunity.uvic.ca/isot/wp-content/uploads/sites/7295/2023/02/ISOT_Fake_News_Dataset_ReadMe.pdf

[16] S. Vosoughi, D. Roy, and S. Aral, “The spread of true and false news online,” Science, vol. 359, no. 6380, pp. 1146–1151, 2018, doi: 10.1126/science.aap9559.

[17] H. Allcott and M. Gentzkow, “Social media and fake news in the 2016 election,” J. Econ. Perspect., vol. 31, no. 2, pp. 211–236, 2017, doi: 10.1257/jep.31.2.211.

[18] D. M. J. Lazer et al., “The science of fake news,” Science, vol. 359, no. 6380, pp. 1094–1096, 2018, doi: 10.1126/science.aao2998.